Linux (Continue)

Shell Scripting III: Environment and Argument

Overview & Learning Outcomes

This lab you will:

1. Review **variables** & **echo**
2. The Environment
3. Access to **arguments**

The Variable

Variables & echo

Creating variables:

- `NAME="Alice"`
- `COUNTER=5`

Variable **NAME**, data is STRING "**Alice**"

Variable **COUNTER**, data is STRING "**5**"

Using variables:

- `echo "Hello $NAME"` # output is "Hello Alice"
- `echo "The counter is $COUNTER"` # output is "The counter is 5"

NOTE: **NO SPACE** around "**=**" in variable assignment

The Environment

What is “**the environment**”?

1. The shell keeps session information called the **environment**
2. Programs can read it to learn **system/user configuration**
3. Some programs use it to **change behavior** (based on variables)
4. You can use it to customize **your shell experience**

Type of Variables

Two main variable types:

1. **Environment Variables**
   - *Or System Environment Variables / Standard Variables*
2. **Shell Variables** (internal to the shell)
   - *This is what we already covered in the variable above*

Standard/System Environment Variables

1. `$HOME` : personal directory
2. `$PATH` : command research path
3. `$TERM` : type of terminal
4. `$IFS` : Input field separators; default is space, tab, and newline
5. `$USER` : current user login
6. `$SHELL` : your shell program name
7. `$PS1` : Primary prompt string ($)
8. `$PS2` : Secondary prompt (>)
9. Type command "`env`" or "`printenv`" to see all environment variables
10. …

Variable Priority Rule

System-wide → User startup files → Current shell session

Variable Priority Rule (Example)

1. Enter "**`env`**"
   - What do you see in "**`$SHELL`**" ?
2. Now, enter "**`SHELL=/bin/sh`**"
3. Then, enter "**`env`**" again
   - What do you see in "**`$SHELL`**" ?
   - Print value of variable "**`$SHELL`**" by using command "**`echo $SHELL`**"
     - *What is the result? (it is the same to what you saw in env command)*
4. Close your terminal and open terminal again
5. Then, enter "**`env`**" again
   - What do you see in "**`$SHELL`**" ? Why your #2 is not saved after you closed the terminal?

The Argument

Arguments

`$0, $1, $2, $3`	: argument in command line
`$?`	: return code by the last command (0 = good)
`$$`	: current process id
`$!`	: last process id that execute in back plan
`$#`	: arguments length
`$*, $@`	: all arguments in the command line

Example:

`./backup_progam.sh -d ./data -b ./backups`

- `$0 = backup_progam.sh`
- `$1 = -d`
- `$# = 4`
- `$* = -d ./data -b ./backups`

Thank You